Les Structures Répétitives en Algorithmique et Langage C : Boucles While, Do-While et For

Thème principal : Structures itératives (boucles while, do-while, for) en algorithmique et langage C

Synthèse exécutive : Ce cours aborde de manière approfondie les trois structures itératives fondamentales (while, do-while, for) en algorithmique et en langage C. L'objectif pédagogique prioritaire est d'apprendre à automatiser l'exécution répétée d'un bloc d'instructions sans duplication de code, en stricte conformité avec le principe DRY (Don't Repeat Yourself). La séance clarifie les critères d'évaluation des conditions booléennes (pré-condition en amont vs post-condition en aval), l'importance de l'initialisation et de la mise à jour des variables de contrôle pour éviter les boucles infinies, ainsi que les règles syntaxiques du C (notamment le point-virgule obligatoire en fin de do-while).

1. Utilité des structures itératives et principe DRY

En programmation algorithmique et en langage C, la répétition d'instructions ne doit jamais être gérée par du copier-coller. Si l'on doit traiter 10 000 éléments d'un tableau ou réitérer une demande de saisie jusqu'à obtenir une valeur valide, dupliquer manuellement les lignes de code détruit la maintenabilité du programme, alourdit inutilement le binaire exécutable et constitue une source majeure de bogues. Les structures itératives (ou boucles) permettent de répéter l'exécution d'un bloc d'instructions tant qu'une condition logique (expression booléenne) reste vraie. L'enjeu central lors de la conception d'une boucle réside dans la maîtrise de la condition d'arrêt et dans l'actualisation effective de la variable de contrôle à chaque itération. Si le corps de la boucle n'altère pas les variables testées dans la condition, celle-ci reste indéfiniment vraie, provoquant une boucle infinie qui gèle l'exécution du programme et sature les ressources du processeur.

- Principe DRY (Don't Repeat Yourself) : Interdiction absolue de dupliquer du code pour réaliser plusieurs fois une même opération.
- Condition d'arrêt : Expression booléenne évaluée à chaque passage pour déterminer la poursuite ou la sortie de la boucle.
- Variable de contrôle / Compteur : Variable modifiée au sein du corps de la boucle pour faire évoluer la condition booléenne vers Faux.
- Boucle infinie : Dysfonctionnement critique surviennent lorsque la condition d'arrêt demeure indéfiniment Vraie (souvent lié à l'omission de la mise à jour de la variable de contrôle).

Cas concret enseignant : *Métaphore du tapis roulant d'usine d'emballage : tant qu'un colis est détecté par le capteur (condition vraie), le bras robotisé l'attrape et l'emballe (instruction). Si le capteur ne détecte plus de colis ou si le quota fixé est atteint, le robot s'arrête. Sans condition de fin fonctionnelle, le robot continuerait à tourner à vide et finirait par surchauffer et détruire la machine.*

2. La boucle TANT QUE (while) : Structure à pré-condition

La structure 'while' est la boucle de référence lorsque le nombre exact d'itérations n'est pas connu à l'avance et dépend d'un événement dynamique externe (saisie utilisateur, fin de lecture d'un fichier, signal réseau). En langage C, sa syntaxe s'écrit `while (condition) { /* instructions */ }`. Le test logique s'effectuant en amont de l'accès au bloc, si la condition s'avère fausse lors du tout premier contrôle, l'intérieur de la boucle est totalement ignoré (0 itération). Il faut porter une attention rigoureuse à l'initialisation des variables de contrôle avant le `while`, ainsi qu'à leur modification explicite à l'intérieur du corps de la boucle pour garantir que la condition s'évaluera à Faux à un moment donné.

- Pré-conditionnement : Évaluation de la condition booléenne AVANT chaque passage dans le bloc d'instructions.
- Plage d'itérations : De 0 à N passages. Si la condition est fausse dès l'entrée, le bloc d'instructions n'est jamais exécuté.

• Initialisation préalable : Toute variable testée dans la condition du while doit impérativement posséder une valeur définie avant d'entrer dans la boucle.

Cas concret enseignant : *Exemple du contrôle du code PIN au distributeur de billets : le distributeur demande le code secret TANT QUE le code saisi est incorrect ET QUE le nombre d'essais est inférieur à 3. Si l'usager tape le bon code dès le premier essai, la boucle s me termine immédiatement sans répéter la demande.*

3. La boucle FAIRE... TANT QUE (do-while) : Structure à post-condition

À la différence de la boucle 'while', la structure 'do-while' exécute toujours les instructions de son corps une première fois avant de vérifier la condition logique de sortie. En langage C, sa syntaxe s'écrit `do { /* instructions */ } while (condition);`. Elle est le choix algorithmique naturel pour les contrôles de saisie (validation de bornes numériques) et la gestion de menus interactifs, où l'action doit être réalisée au moins une fois avant de demander si l'utilisateur souhaite réitérer. L'oubli du point-virgule final constitue l'une des erreurs de syntaxe les plus fréquentes commises lors des évaluations sur machine et provoque un refus de compilation.

• Post-conditionnement : Évaluation du test logique APRÈS l'exécution du bloc d'instructions.

• Garantie d'exécution : La boucle s'exécute obligatoirement au minimum 1 fois (intervalle de 1 à N itérations).

• Syntaxe C stricte : Obligation impérative de placer un point-virgule ';' immédiatement après la parenthèse fermante du while : `do { ... } while(condition);`.

Cas concret enseignant : *Contrôle de saisie d'une note d'examen : 'Il est techniquement impossible de vérifier si la saisie faite par un étudiant est valide avant qu'il n'ait tapé au moins une valeur sur le clavier. On doit donc exécuter la saisie au moins une fois, ce qui fait du do-while la structure naturellement adaptée.'*

4. La boucle POUR (for) : Itération déterminée

La structure 'for' rationalise la mécanique itérative en réunissant sur une seule ligne d'en-tête les trois étapes indispensables à la répétition : 1) l'initialisation de la variable de contrôle (ex: `i = 1`), 2) la condition logique de poursuite (ex: `i <= n`), et 3) la mise à jour (incrémentation ou décrémentation, ex: `i++`). En langage C, la syntaxe s'écrit `for (init; cond; inc) { /* instructions */ }`. Bien que le langage C permette syntaxiquement d'omettre certaines expressions dans l'en-tête, les renseigner explicitement est une exigence absolue de lisibilité algorithmique et de propreté logicielle.

• En-tête condensé : Centralise l'initialisation, la condition de maintien et la mise à jour sous la forme `for(init; cond; inc)`.

• Nombre d'itérations déterminé : Structure réservée aux situations où le nombre exact de répétitions est connu avant d'entrer dans la boucle.

• Portée (scope) : Possibilité de déclarer la variable de contrôle directement dans l'en-tête (ex: `for(int i = 0; ...)`).

Cas concret enseignant : *Métaphore des tours de piste d'athlétisme : 'Vous savez avant de vous lancer que vous devez effectuer exactement 5 tours. Vous initialisez votre compteur au tour 1 (init), vous vérifiez que vous n'avez pas dépassé 5 (cond), et à chaque franchissement de la ligne d'arrivée, vous ajoutez 1 à votre compteur (inc).'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le programme plante ou bloque indéfiniment quand j'oublie d'incrémenter 'i' dans une boucle while ?
Réponse du professeur : Si la variable de contrôle 'i' n'est pas mise à jour à l'intérieur du corps de la boucle, la condition d'arrêt reste toujours vraie. Le programme entre alors dans une boucle infinie : il répète indéfiniment les mêmes instructions et bloque l'exécution en consommant l'ensemble des ressources du processeur.

Question étudiant : Est-ce qu'on peut toujours remplacer une boucle 'for' par une boucle 'while' ?
Réponse du professeur : Oui, sur le plan logique la boucle 'while' est plus générale. Une structure `for(init; cond; inc)` peut toujours se réécrire avec un `while` en plaçant `init` avant la boucle, `cond` dans l'en-tête du `while`, et `inc` à la toute fin du bloc d'instructions. Cependant, on privilégie la boucle 'for' dès que le nombre d'itérations est déterminé à l'avance pour des raisons d'explicitation et de lisibilité du code.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme complet en langage C qui calcule la somme des N premiers entiers positifs (ex: si N = 5, calculer 1 + 2 + 3 + 4 + 5 = 15). Le programme doit valider que N est un entier strictement positif (> 0) à l'aide d'une boucle 'do-while', puis effectuer le calcul cumulatif au moyen d'une boucle 'for'. Le code doit respecter la norme de codage de l'IUT d'Anglet (cartouche, tripartition, flèches de flot, style Allman).

Démarche de résolution :
Étape 1 : Déclarer les variables `n`, `somme` et `i` dans la section // VARIABLES. Étape 2 : Mettre en place une boucle `do-while` qui demande la saisie de `n` et réitère tant que `n <= 0` afin de garantir qu'aucun nombre négatif ou nul ne passe. Étape 3 : Initialiser `somme = 0` avant d'effectuer les cumuls. Étape 4 : Utiliser une boucle `for` allant de `i = 1` jusqu'à `i = n` avec un pas de `1`, en ajoutant `i` à `somme` à chaque itération. Étape 5 : Afficher le résultat formaté avec `printf`.

```
/*
  En-tête conforme IUT Bayonne et Pays Basque - Campus Anglet
  Nom : Matéi
  Matière : R1.01 - Initiation à l'Algorithmique et la Programmation
  Objectif : Calcul de la somme des N premiers entiers avec validation de saisie
*/

#include <stdio.h>
#include <stdlib.h>

int main()
{
    // CONSTANTES

    // VARIABLES
    int n;          // Nombre d'entiers à sommer (saisi par l'utilisateur)
    int somme;      // Accumulateur pour le calcul de la somme
    int i;          // Variable de contrôle de la boucle for

    // TRAITEMENTS
    // Étape 1 : Saisie sécurisée de N avec la structure do-while
    do
    {
        printf("Entrez un nombre entier N strictement positif (> 0) : ");
        // (clavier) >> saisir >> n
        scanf("%d", &n);
    }
    while (n <= 0);

    // Étape 2 : Initialisation de l'accumulateur
    somme = 0;

    // Étape 3 : Calcul cumulatif avec la boucle for
    for (i = 1; i <= n; i++)
    {
        somme = somme + i;
    }

    // Étape 4 : Affichage du résultat final
    printf("La somme des %d premiers entiers est : %d\n", n, somme);

    return 0;
}
```

Résultat : Pour une saisie utilisateur où N = 5 : La boucle 'do-while' valide N (> 0). La boucle 'for' effectue 5 itérations : à i=1 (somme=1), i=2 (somme=3), i=3 (somme=6), i=4 (somme=10), i=5 (somme=15). Le programme affiche correctement : 'La somme des 5 premiers entiers est : 15'. Si l'utilisateur saisit d'abord -3, le do-while rejette la saisie et demande un nouvel entier.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'évaluation entre une boucle 'while' et une boucle 'do-while' ?

R : La boucle 'while' est à pré-condition : la condition est testée AVANT d'entrer dans la boucle (0 à N itérations). La boucle 'do-while' est à post-condition : la condition est testée APRÈS l'exécution du bloc (au moins 1 itération garantie).

Q : Que se passe-t-il si la condition initiale d'une boucle 'for' est fausse dès le premier contrôle ?

R : Le corps de la boucle 'for' n'est jamais exécuté (0 itération), et le programme poursuit directement son exécution à l'instruction qui suit la boucle.

Q : Quel est l'oubli de syntaxe le plus fréquent qui provoque une erreur de compilation sur la boucle 'do-while' en C ?

R : L'omission du point-virgule final ';' immédiatement après la parenthèse fermante de la condition `while (condition);`.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765